MOSS DIGITAL · BIBLIOTECA PRÁTICA

AMOSTRA GRATUITA · MÓDULO 01

Programação do Zero.

Seu Primeiro Projeto Web

Fundamentos, exemplos comentados e exercícios para aprender construindo.

Obra-base: Prof. Marcelo Oliveira

Projeto de exemplo: Hamburgueria Prime

Português brasileiro · Edição digital ampliada, 2026

Conheça antes de comprar

Esta amostra contém o módulo de Fundamentos da Web. O livro completo reúne 74 aulas em 15 módulos e acompanha uma versão HTML de consulta.

Valor do livro: R$ 29,90. Atendimento e entrega manual pelo WhatsApp da Moss Digital: (11) 94203-8103.

Projeto de exemplo: Hamburgueria Prime. Leia, experimente as práticas e avalie se o formato escrito combina com seu jeito de estudar.

MÓDULO 01 · 4 AULAS · 4H 10MIN

Fundamentos da Web

Ao final deste módulo o estudante explica como um pedido viaja da barra do navegador até o servidor e de volta, organiza a pasta do projeto Hamburgueria Prime com nomes previsíveis, escreve algoritmos em pseudocódigo antes de programar e aplica critérios básicos de acessibilidade e ética digital ao planejar o cardápio online.

OBJETIVO

compreender o funcionamento da web e preparar o terreno mental e físico (arquivos, lógica, princípios) antes da primeira linha de HTML.

PRÉ-REQUISITOS

nenhum: basta ter um navegador instalado e acesso a um editor de texto.

RESULTADO NO PROJETO

pasta `hamburgueria-prime/` criada com estrutura de pastas, um esboço de cardápio em pseudocódigo e uma lista de decisões de acessibilidade/ética do projeto.

NESTE MÓDULO

1. Aula 01 — Internet, navegador e servidor
2. Aula 02 — Ferramentas e organização
3. Aula 03 — Algoritmos e lógica
4. Aula 04 — Acessibilidade e ética digital

Orientação docente do módulo

Este módulo é o mais abstrato do curso: nada dele aparece na tela do projeto, e é justamente aí que turmas iniciantes costumam achar que "não serve para nada". Conecte cada aula a um artefato concreto da Hamburgueria Prime — o desenho da requisição vira a planta do site, a árvore de pastas vira o repositório, o pseudocódigo vira o carrinho futuro. Reserve um tempo extra na aula 03 para quem nunca escreveu uma condição; para turmas avançadas, peça que prevejam o que acontece com a página quando o servidor responde 404. Antes de avançar para o Módulo 02, confirme que todos têm a pasta do projeto criada e os critérios de acessibilidade anotados — o HTML das próximas aulas será escrito dentro dessa estrutura.

AULA 01

Internet, navegador e servidor

Iniciante | 50 min | Projeto: planta da requisição

PARA O PROFESSOR

1. **Diagnóstico (5 min):** pergunte: "o que acontece entre você digitar `hamburgueriaprime.com` e a imagem do Prime Clássico aparecer?" Anote as hipóteses no quadro, sem corrigir — você vai voltar nelas no fechamento.
2. **Demonstração (10 min):** abra o DevTools (aba Network), carregue um site e mostre as linhas de requisição com status 200. Faça uma requisição com o Wi-Fi desligado para exibir o erro de rede e discuta a diferença entre "servidor respondeu com erro" e "não conseguiu nem falar com o servidor".
3. **Programação guiada (15 min):** desenhem juntos, no quadro, a viagem do pedido do cardápio: navegador → DNS → servidor → resposta HTML/CSS/JS → navegador renderiza.
4. **Prática individual ou em duplas (15 min):** cada dupla reproduz o esquema comentado abaixo em uma folha, marcando em vermelho onde um erro de rede interromperia o caminho.
5. **Fechamento (5 min):** pergunte: "domínio e hospedagem são a mesma coisa?" e registre a explicação de um aluno no caderno da turma.

> **Dica:** não gaste tempo decorando siglas — o que importa é a ordem dos papéis (navegador pede, servidor responde). A aba Network pertence às ferramentas de desenvolvedor no computador; o Chrome no Android não oferece essa mesma aba diretamente. Se a turma tiver só celular, use o desenho em papel: a prática da aula é desenho, não instalação.

EXPLICAÇÃO

A internet e a web parecem a mesma coisa, mas são camadas diferentes. A internet é a infraestrutura: cabos, roteadores, provedores e uma malha de redes interligadas em todo o mundo. A web (World Wide Web) é um serviço que roda por cima dessa infraestrutura e troca documentos usando o protocolo HTTP ou HTTPS. Quando você diz "abrir um site", está usando um aplicativo (o navegador) que fala HTTP com outra máquina (o servidor) através da internet. Separar esses dois conceitos ajuda a entender problemas reais: se a internet está fora do ar, nenhum site abre; se só um site está fora, o problema está no servidor dele.

O ciclo básico tem dois tempos. Primeiro o navegador envia uma **requisição** (request) dizendo o que quer — por exemplo, a página do cardápio da Hamburgueria Prime. Depois o servidor devolve uma **resposta** (response) com um código de status e o conteúdo: HTML para estrutura, CSS para aparência, JavaScript para comportamento. O navegador recebe tudo isso em texto puro e transforma na página que você vê. Esse texto é tão legível que qualquer pessoa pode inspecionar o que um site devolveu — o que é ótimo para estudar e também um lembrete de que nada que chega ao navegador é segredo.

Antes de a requisição chegar ao servidor, porém, é preciso descobrir o endereço. O nome digitado na barra (como `hamburgueriaprime.com`) é um domínio, e o DNS pode associar esse nome a um ou mais endereços IP. É como ligar para uma loja: você procura o nome na agenda (DNS), mas a ligação usa um número (IP). Domínio e hospedagem não são a mesma coisa: o domínio é o nome que o cliente digita; a hospedagem é o serviço que disponibiliza os arquivos ou a aplicação na rede. A disponibilidade depende do serviço contratado e de seu funcionamento. Você pode apontar um domínio para outra hospedagem, e uma hospedagem pode atender vários domínios.

O "S" do HTTPS significa segurança: por trás do HTTP comum há o TLS, uma camada de criptografia que impede que alguém no meio da rede leia ou altere o que trafega — dados de login, telefone do cliente, endereço de entrega. Para uma hamburgueria fictícia o risco é baixo, mas o hábito de publicar tudo em HTTPS já na primeira hospedagem evita retrabalho: certificado grátis, configuração em minutos e nenhum aviso de "site não seguro" assustando o cliente.

Por que isso importa na vida prática? Porque quase todo bug de site se classifica em três perguntas: o navegador conseguiu falar com o servidor? o servidor entendeu o que foi pedido e respondeu certo? o navegador interpretou corretamente o que chegou? Aprender a separar esses três momentos transforma "o site não abre" em um diagnóstico minúsculo e verificável. No projeto Hamburgueria Prime, o esquema que você desenhar nesta aula vira a planta de tudo o que virá: rotas, arquivos estáticos, imagens do cardápio e, mais para frente, o pedido enviado ao servidor.

VOCABULÁRIO TÉCNICO

TERMO (EN)	SIGNIFICADO (PT-BR)
`URL` (Uniform Resource Locator)	endereço completo que identifica um recurso, como `https://hamburgueriaprime.com/cardapio`.
`HTTP` (HyperText Transfer Protocol)	protocolo de requisição e resposta usado para trocar páginas e dados na web.
`HTTPS` + `TLS`	HTTP criptografado: impede leitura e adulteração do tráfego no caminho.
`DNS` (Domain Name System)	serviço que traduz nomes de domínio em endereços IP numéricos.
`server` (servidor)	máquina que fica disponível o tempo todo e responde requisições.
`request / response`	requisição/resposta: os dois tempos de toda conversa entre navegador e servidor.
`status code`	código numérico do resultado (200 ok, 404 não encontrado, 500 erro do servidor).
`hosting` (hospedagem)	espaço em servidor onde os arquivos do site ficam publicados; diferente do domínio.

CÓDIGO COMENTADO

```
VIAGEM DE UM PEDIDO AO CARDÁPIO (esquema comentado)

 [1] Usuário digita: hamburgueriaprime.com
        |
 [2] Navegador pergunta ao DNS: "onde fica esse nome?"
        |   DNS responde com um número IP (o endereço real)
        v
 [3] Navegador monta a REQUISIÇÃO
        GET /cardapio  HTTP/2
        Host: hamburgueriaprime.com
        |
        |  --- HTTPS protege este trecho com TLS ---
        v
 [4] Servidor recebe, procura o arquivo e monta a RESPOSTA
        Status: 200 OK
        Corpo: <html> ... cardápio ... </html>   (mais CSS e JS)
        |
        v
 [5] Navegador lê o texto e desenha a página na tela

CASOS DE ERRO (o mesmo caminho, com desvio):
 404 → caminho existe na internet, mas o arquivo não existe no servidor
 500 → o servidor entendeu o pedido e quebrou tentando atendê-lo
 offline → falha no passo [2] ou [3]: nem chegou a falar com o servidor
```

Leitura linha a linha

1. `[1] Usuário digita: hamburgueriaprime.com` — o ponto de partida é sempre um nome amigável; sem isso, só existiriam números IP difíceis de memorizar.
2. `[2] Navegador pergunta ao DNS` — se o DNS estiver fora do ar, o navegador falha aqui mesmo com a internet "funcionando"; por isso "site não abre" nem sempre é culpa do servidor.
3. `GET /cardapio` — método + caminho: diz ao servidor *o que* quer, sem ainda dizer *quem* pediu.
4. `Host: hamburgueriaprime.com` — um mesmo servidor pode hospedar vários sites; o cabeçalho Host diz qual deles deve atender.
5. `HTTPS protege este trecho com TLS` — marca onde a criptografia começa: tudo depois daqui viaja ilegível para olhos estranhos.
6. `Status: 200 OK` — o servidor sempre responde com um código, mesmo quando a resposta for "não achei"; o navegador decide o que mostrar a partir dele.
7. `Corpo: <html> ... </html>` — o que chega é texto puro; nenhum pixel vem pronto, o navegador reconstrói a imagem.

8. `[5] Navegador lê o texto e desenha` — renderização: interpretar HTML, aplicar CSS e executar JS; problemas de "como ficou" nascem aqui.
9. `404 / 500 / offline` — os três desvios cobrem quase toda dúvida inicial de um aluno sobre "por que não carrega".

PRÁTICA GUIADA

1. Abra o DevTools do navegador (F12) e a aba **Network** em um site qualquer.
2. Recarregue a página e observe as linhas: clique na primeira e localize o campo *Status Code* e o tipo *document*.
3. Identifique quais linhas são HTML, CSS, JS e imagens — são as "respostas" separadas do passo [4].
4. Repita o esquema de 5 passos do código acima em uma folha, substituindo o domínio fictício por `hamburgueriaprime.com` e marcando em vermelho onde a falha de rede interromperia o caminho.
5. Desative a conexão (ou use o modo offline do DevTools) e recarregue: compare a mensagem com um erro 404 obtido digitando um endereço errado.
6. **Resultado esperado:** um diagrama de 5 passos anotado, mais a constatação registrada de que "offline" falha antes do servidor e "404" falha no servidor — evidência: captura da aba Network e foto/desenho do diagrama.

EXERCÍCIOS

1. **FIXAÇÃO** Por que domínio e hospedagem não são a mesma coisa? Dê um exemplo de um sem o outro.

   ▼ Gabarito e dica

   O domínio é o nome registrado (o que o cliente digita); a hospedagem é o espaço físico em servidor onde os arquivos moram. Exemplo: registrar `hamburgueriaprime.com` e apontar para a hospedagem de outra empresa; ou contratar hospedagem que já entrega um endereço temporário sem domínio próprio. Dica: pense na diferença entre o nome de uma loja na rua e o galpão onde a mercadoria fica.

2. **APLICAÇÃO** Descreva, com suas próprias palavras, a viagem de um pedido do cardápio (Prime Clássico, R$ 29,90) desde a barra do navegador até o botão "Adicionar ao pedido" aparecer na tela.

   ▼ Gabarito e dica

   Sequência esperada: digitação do domínio → consulta DNS → requisição HTTPS GET → servidor responde 200 com HTML, CSS e JS → navegador renderiza e, ao clicar, o JavaScript manipula a página. Dica: use os 5 passos do esquema e cite o status 200 ao menos uma vez.

3. **FIXAÇÃO** Um amigo diz: "meu site caiu porque meu domínio venceu". Explique se essa afirmação faz sentido e o que provavelmente aconteceu.

▼ Gabarito e dica

Faz sentido, mas precisa de uma verificação: se o domínio venceu, o DNS pode deixar de responder e o site "some" mesmo com o servidor de pé; se a hospedagem venceu, o servidor responde com erro ou não responde. Dica: pergunte qual dos dois (nome ou galpão) foi renovado — o teste é tentar abrir por IP ou checar o status do servidor.

4. **DESAFIO** Explique a diferença entre os erros 404 e 500 usando a analogia do restaurante, e indique em qual etapa do esquema cada um acontece.

▼ Gabarito e dica

404 = o restaurante recebeu seu pedido, procurou no cardápio e não existe (o caminho está errado ou o arquivo não está no servidor) — acontece no passo [4]. 500 = o pedido foi recebido, mas a cozinha quebrou tentando preparar (erro no código do servidor) — também no [4], porém por falha interna. Dica: nenhum dos dois é problema do cabeçalho do restaurante (DNS) nem da estrada (internet).

ERROS COMUNS

- **Confundir internet com web:** o aluno diz "a internet está lenta" quando o problema é o servidor do site. Corrija pedindo que ele aponte em qual etapa do esquema a lentidão aparece.
- **Achar que o navegador "baixa o site pronto":** ele recebe texto e desenha; por isso a mesma página pode parecer diferente em navegadores diferentes. Mostre o HTML na aba Elements.
- **Tratar 404 como falha de internet:** 404 significa que a conexão funcionou e o servidor respondeu. Contraste com o modo offline na demonstração.
- **Esquecer o HTTPS desde a hospedagem:** alunos abrem o site pela primeira vez e ignoram o aviso "não seguro". Combine que todo exemplo do curso será publicado com HTTPS.
- **Copiar o endereço do navegador errado:** digitam a rota sem o domínio ou colam caminho de arquivo local (`file:///`) em discussões sobre servidor — valide sempre o formato completo da URL.

CRITÉRIOS DE VERIFICAÇÃO

- ✓ O estudante explica com as próprias palavras a diferença entre internet, web, navegador e servidor, indicando quem pede e quem responde.
- ✓ O diagrama de 5 passos está completo, com o caminho do cardápio da Hamburgueria Prime e os desvios 404/500/offline marcados.
- ✓ A atividade tem evidência verificável: captura da aba Network e o desenho/registro do esquema.

✓ O estudante aponta uma limitação ou cuidado: por exemplo, o que um atacante veria sem HTTPS ou por que dados do cliente não devem trafegar em claro.

← Início do módulo

Próxima aula: Ferramentas e organização →

AULA 02

Ferramentas e organização

Iniciante | 50 min | Projeto: árvore de pastas

PARA O PROFESSOR

1. **Diagnóstico (5 min):** pergunte: "onde vocês guardariam o arquivo da foto do Prime Cheddar em um projeto com 50 imagens?" Anote as respostas — provavelmente virá "na mesma pasta do HTML", o que você vai rebater na demonstração.
2. **Demonstração (10 min):** crie a árvore de pastas ao vivo no editor, mostrando o painel lateral. Renomeie `style.css` para `estilo.css` sem atualizar o HTML e abra a página: o link quebrado é o erro proposital da aula.
3. **Programação guiada (15 min):** a turma digita junto a estrutura completa, comentando a responsabilidade de cada pasta em voz alta.
4. **Prática individual ou em duplas (15 min):** cada um monta a árvore do zero em uma pasta vazia e cria quatro arquivos placeholder (`index.html`, `css/style.css`, `js/app.js`, `assets/img/`).
5. **Fechamento (5 min):** pergunte: "qual problema surge ao renomear `style.css` sem atualizar o HTML?" e confirme que todos salvaram a árvore como evidência.

> **Dica:** nomes em minúsculas sem acento não são preciosismo: servidores Linux diferenciam `Cardapio.HTML` de `cardapio.html`, e acentos podem virar percent-encoding quebrado no caminho. Se a turma usar apenas celular, monte a mesma árvore em um gerenciador de arquivos do Android/iOS e registre um print — o conceito é o mesmo.

EXPLICAÇÃO

Antes de escrever código, vale escrever o endereço do código. Um projeto bem organizado é aquele em que qualquer pessoa — inclusive você, três semanas depois — abre a pasta e sabe onde procurar. A regra prática é simples: cada tipo de arquivo mora em um lugar com nome claro. HTML fica na raiz porque é a porta de entrada; CSS fica em `css/`, JavaScript em `js/` e imagens em `assets/img/`. Essa separação não é burocracia: ela permite que você entregue só as imagens para um designer, troque todo o CSS sem tocar no HTML e encontre um arquivo quebrado em segundos.

Os nomes importam tanto quanto as pastas. Use minúsculas, sem acento, sem espaço e com hífen como separador (`prime-cheddar.webp`): espaços viram `%20` nos endereços, acentos podem quebrar

dependendo da hospedagem, e maiúsculas em algum ponto do caminho geram erro 404 em servidores Linux, que diferenciam `Pedido.html` de `pedido.html`. Escolha também extensões coerentes: `.html` para página, `.css` para estilo, `.js` para comportamento, `.webp` ou `.png` para imagem. Previsibilidade é a palavra-chave — em time, ninguém precisa perguntar "onde fica isso?".

A ferramenta central é o editor. O VS Code combina arquivo, painel de pastas, terminal integrado e busca global; para quem está começando, o fato de o editor e o terminal morarem na mesma janela elimina uma confusão clássica de "em qual pasta estou rodando o comando?". Existem alternativas válidas (Sublime, Notepad++, até apps de celular como o Acode), e o que importa não é a marca, e sim três capacidades: abrir pastas inteiras, mostrar erros de digitação de HTML/CSS e permitir testar o resultado rapidamente no navegador.

Outra peça-chave é o servidor local. Abrir o arquivo com duplo-clique (`file:///`) funciona para páginas soltas, mas algumas coisas só se comportam direito servidas por HTTP: caminhos relativos profundos, módulos de JavaScript, requisições futuras a APIs. Um servidor local resolve com um comando (`npx serve`, `python -m http.server` ou a extensão *Live Server*) e reproduz mais fielmente o que acontecerá na hospedagem — inclusive o modo como o navegador resolve os caminhos que você estudará na Aula 09. Testar sempre no servidor local evita a surpresa de "funciona no meu computador e não publica".

Por fim, organize também o seu processo: trabalhe em uma pasta só por projeto, mantenha o arquivo principal sempre chamado `index.html` (é o padrão que qualquer hospedagem procura automaticamente) e nunca misture o projeto do curso com downloads ou trabalhos antigos na mesma pasta. Se algo der errado — e vai dar —, uma árvore limpa permite desfazer sem medo. Ao final desta aula, a Hamburgueria Prime terá exatamente a estrutura que receberá HTML, CSS e imagens até o fim do curso.

VOCABULÁRIO TÉCNICO

TERMO (EN)	SIGNIFICADO (PT-BR)
`IDE` (Integrated Development Environment)	ambiente de desenvolvimento: editor + terminal + ferramentas na mesma janela.
`file / folder`	arquivo (documento único) e pasta (diretório que agrupa arquivos).
`project root`	raiz do projeto: pasta principal que contém tudo o que pertence a ele.
`relative path`	caminho relativo: endereço de um arquivo contado a partir da posição atual.
`local server`	servidor local: programa que serve os arquivos do seu computador por HTTP.
`extension`	extensão: sufixo do nome (`.html`, `.css`, `.js`) que indica o tipo.

TERMO (EN)	SIGNIFICADO (PT-BR)
`placeholder`	arquivo provisório criado só para existir e ser referenciado enquanto se monta o resto.
`versioning` (versionamento)	controle de versões: registrar mudanças para poder voltar atrás (Git, ver módulo 07).

CÓDIGO COMENTADO

```
hamburgueria-prime/            ← raiz do projeto: tudo mora aqui
  index.html                   ← porta de entrada (padrão de toda hospedagem)
  css/
    style.css                  ← TODO o visual da página fica neste arquivo
  js/
    app.js                     ← comportamento: carrinho, cliques, cálculo
  assets/
    img/
      classico.webp            ← Prime Clássico (R$ 29,90)
      cheddar.webp             ← Prime Cheddar (R$ 34,90)
      veggie.webp              ← Prime Veggie (R$ 32,90)
      logo.png                 ← marca da hamburgueria
  README.md                    ← descrição curta do projeto (opcional)

NOMENATURA (combinação que funciona em qualquer hospedagem):
  minúscula + sem acento + sem espaço + hífen separando palavras
  certo: prime-cheddar.webp   errado: Prime Cheddar (2).WEBP

ONDE CADA LINK VAI APONTAR (referência para as próximas aulas):
  index.html ──> css/style.css       (dentro do projeto)
  index.html ──> js/app.js           (dentro do projeto)
  index.html ──> assets/img/*.webp   (dentro do projeto)
```

Leitura linha a linha

1. `hamburgueria-prime/` — a raiz: todos os caminhos relativos serão contados a partir dela; guardar outro projeto dentro daqui mistura tudo.
2. `index.html` — nome especial: hospedagens procuram esse arquivo automaticamente ao abrir o domínio; renomeá-lo para `pagina.html` faz o site "sumir" da raiz.
3. `css/style.css` — uma pasta para estilos evita que a raiz fique cheia de arquivos e permite trocar o visual inteiro sem abrir o HTML.
4. `js/app.js` — comportamento separado da estrutura: o HTML continua legível mesmo quando a lógica cresce.

5. `assets/img/` — imagens em subpasta profunda: pasta nome `assets` é um padrão de mercado que qualquer colega reconhece.
6. `classico.webp / cheddar.webp / veggie.webp` — nomes previsíveis ligados ao cardápio; achar a foto do item vira busca de 2 segundos.
7. `minúscula + sem acento + sem espaço` — a regra que evita 404 silencioso em servidores Linux e URLs com `%20`.
8. `index.html ⟶ css/style.css` — mostra que o HTML referenciará esses caminhos; quando um nome mudar, a referência quebra (erro da demonstração).

PRÁTICA GUIADA

1. Crie a pasta `hamburgueria-prime` em um local fácil de encontrar (não na Área de Trabalho de outra pessoa, não dentro de Downloads de curso antigo).
2. Abra essa pasta **inteira** no editor (arquivo → abrir pasta), não um arquivo avulso.
3. Crie a árvore exata do código acima: `index.html`, `css/style.css`, `js/app.js` e `assets/img/`.
4. Escreva em `style.css` uma regra qualquer (ex.: `body { font-family: sans-serif; }`) e em `index.html` a etiqueta `<link rel="stylesheet" href="css/style.css">`.
5. Renomeie `style.css` para `estilo.css` e recarregue a página sem tocar no HTML: observe o visual sumir.
6. Corrija o caminho no HTML (ou desfaça o renomeio) e confirme que o estilo voltou.
7. **Resultado esperado:** árvore de pastas visível no painel do editor, página abrindo no navegador e a sequência "quebrou → corrigido" registrada em capturas; evidência: print da árvore + print da página com o estilo restaurado.

EXERCÍCIOS

1. **FIXAÇÃO** Qual problema surge ao renomear `style.css` sem atualizar o HTML? Como o erro aparece para o usuário final?

   ▼ Gabarito e dica

   A etiqueta `<link>` aponta para um arquivo que não existe mais: o navegador recebe 404 na folha de estilos e renderiza o HTML sem nenhuma formatação — texto preto, sem cor, sem fonte. O usuário vê "o site quebrou", mas o HTML continua funcionando. Dica: o DevTools (aba Network) mostra o 404 do CSS.

2. **APLICAÇÃO** Monte a árvore de pastas da Hamburgueria Prime e grave ao lado de cada pasta a responsabilidade dela em uma frase.

   ▼ Gabarito e dica

   Árvore + anotações: raiz = projeto inteiro; `index.html` = entrada; `css/` = visual; `js/` = comportamento; `assets/img/` = fotos do cardápio (classico, cheddar, veggie, logo). Dica: se você não souber justificar a existência de uma pasta, ela provavelmente não deveria existir ainda.

3. **FIXAÇÃO** Por que nomes de arquivo devem ser minúsculos, sem acento e sem espaço?

   ▼ Gabarito e dica

   Servidores Linux diferenciam maiúsculas de minúsculas (`Cardapio.html` ≠ `cardapio.html`), acentos e espaços exigem codificação na URL e podem falhar dependendo da hospedagem — o resultado é um 404 que não existe no seu computador. Dica: pense no nome como um endereço que trafega por linhas de comando e redes do mundo todo.

4. **DESAFIO** Um colega enviou um projeto com tudo solto na raiz e 47 imagens chamadas `Imagem1.png` … `Imagem47.png`. Proponha um plano de reorganização e explique o que precisará ser atualizado em seguida.

   ▼ Gabarito e dica

   Plano esperado: criar `assets/img/`, mover as imagens, renomear de forma previsível ligada ao conteúdo (ex.: `prime-cheddar.webp`), depois atualizar todos os `src` do HTML e conferir um a um no navegador. Dica: mover arquivo nunca é automático em relação às referências — quem muda de lugar é o arquivo, quem muda de endereço é o HTML.

ERROS COMUNS

- **Abrir o arquivo avulso em vez da pasta:** o painel de pastas fica vazio e buscas globais não funcionam. Corrija reabrindo a raiz do projeto.
- **Gravar o projeto em `Downloads` ou na Área de Trabalho:** limpezas do sistema apagam o trabalho. Combine uma pasta fixa, ex.: `Documentos/cursos/hamburgueria-prime`.
- **Copiar o caminho absoluto do computador para o HTML (`C:\Users\ … \style.css`):** funciona só na sua máquina e quebra na hospedagem. Use sempre caminho relativo.
- **Espaços e acentos nos nomes:** `Prime Cheddar Final 2.png` gera URL codificada e confusão. Renomeie antes de referenciar.
- **Trocar a extensão pelo editor errado:** salvar `style.css` como `style.css.txt` faz o navegador ignorar o estilo. Confira a extensão real no painel de arquivos.

CRITÉRIOS DE VERIFICAÇÃO

- ✓ O estudante justifica em uma frase a função de cada pasta da árvore criada.
- ✓ A pasta `hamburgueria-prime` existe com `index.html`, `css/`, `js/` e `assets/img/` — evidência: captura do painel do editor.
- ✓ O estudante reproduziu o erro do arquivo renomeado e o corrigiu, registrando as duas capturas.

✓ O estudante aponta uma limitação/cuidado: caminhos relativos dependem da localização do arquivo (ou: servidor Linux difere maiúsculas de minúsculas).

← Aula anterior: Internet, navegador e servidor

Próxima aula: Algoritmos e lógica →

AULA 03

Algoritmos e lógica

Iniciante | 50 min | Projeto: cálculo do pedido em pseudocódigo

---

PARA O PROFESSOR

1. **Diagnóstico (5 min):** escreva no quadro "como preparar um café?" e peça que a turma dê instruções literalmente — depois mostre como uma instrução ambígua ("coloque um pouco de açúcar") quebra a execução.
2. **Demonstração (10 min):** transforme a receita em algoritmo com variável, condição e repetição, cometendo de propósito um erro (esquecer o fim do `se`) para mostrar como a leitura fica impossível.
3. **Programação guiada (15 min):** escrevam juntos o algoritmo do pedido da Hamburgueria Prime em pseudocódigo, linha a linha, antes de qualquer sintaxe.
4. **Prática individual ou em duplas (15 min):** cada dupla escreve o algoritmo de somar itens do cardápio e roda a "simulação" apontando o dedo nas linhas com dois casos de teste.
5. **Fechamento (5 min):** pergunte: "quando uma condição é mais adequada que uma repetição?" e valide respostas com exemplos do cardápio.

> **Dica:** proiba o compilador nesta aula — quem abre o editor cedo demais começa a brigar com sintaxe e deixa de pensar na lógica. O pseudocódigo é deliberadamente informal: se dois alunos escreverem de forma diferente e ambas resolverem o problema, está certo.

EXPLICAÇÃO

Um algoritmo é uma sequência **finita** de passos que resolve um problema — finita no duplo sentido: começa, termina e não repete para sempre. Você já escreveu milhares de algoritmos sem perceber: uma receita, o roteiro até o ponto de ônibus, a lista de tarefas da manhã. Programar é o mesmo exercício, com uma diferença rigorosa: a máquina não infere nada. Onde uma pessoa completa automaticamente o "coloque um pouco de açúcar", o computador exige valor, unidade e critério. Essa exigência é o que parece frieza no início e se revela, depois, uma disciplina que evita bugs caros.

Três construções cobrem quase tudo o que um iniciante precisa. **Variáveis** guardam valores que mudam ao longo da execução — `quantidade`, `total`, `nomeDoItem`. **Condições** decidem um caminho com base em uma pergunta de verdadeiro/falso — `se quantidade > 0 então`. **Repetições** percorrem coleções ou

repetem enquanto algo for verdade — `para cada item do pedido faça`. Repare na hierarquia de aprendizado: dá para simular um pedido com apenas uma condição; a repetição só se torna necessária quando o número de itens é desconhecido — e é exatamente isso que acontece com um carrinho de compras real.

O pseudocódigo é a ferramenta de transição entre a língua e a linguagem de programação. Ele usa palavras quase naturais (`INÍCIO`, `ler`, `se`, `então`, `FIM`) com indentação para mostrar o que está dentro de quê. Escrever em pseudocódigo força duas decisões que o código pronto costuma esconder: quais dados eu preciso receber e o que exatamente muda em cada passo. Quando o aluno tenta pular direto para JavaScript, ele costuma errar justamente aí — não sabe o que o programa deve receber nem qual é a resposta esperada.

Testar é parte do algoritmo, não um extra. Antes de rodar, escolha casos de teste: um caso normal (1 × Prime Clássico), um caso-limite (quantidade 0) e um caso inválido (número negativo). Simule o algoritmo na mão, passo a passo, e compare com o resultado que você esperaria na vida real. Essa prática — executar o código no papel — é o que separa quem entende do que quem digita: quando o resultado divergir, você já sabe em qual linha a história mudou de direção.

No projeto Hamburgueria Prime, o algoritmo que você escrever hoje antecipa o carrinho das próximas aulas: receber itens, validar quantidade, calcular subtotal, aplicar regra de entrega e mostrar o total. Cada uma dessas frases vira linhas de código depois. Guardar o pseudocódigo no repositório (até como comentário no próprio arquivo) também ajuda o professor a avaliar o raciocínio separadamente da sintaxe — erros de vírgula não mascaram mais erros de lógica.

VOCABULÁRIO TÉCNICO

TERMO (EN)	SIGNIFICADO (PT-BR)
`algorithm`	algoritmo: sequência finita e ordenada de passos que resolve um problema.
`variable`	variável: nome que guarda um valor que pode mudar durante a execução.
`if / else`	se / senão: estrutura condicional que escolhe entre caminhos.
`loop`	repetição: estrutura que executa um bloco várias vezes ou enquanto houver condições.
`pseudocode`	pseudocódigo: escrita informal entre a língua e a linguagem, para planejar antes de codar.
`input / output`	entrada/saída: o que o algoritmo recebe do mundo e o que ele devolve.
`test case`	caso de teste: conjunto de entrada e resultado esperado usado para validar o algoritmo.
`indentation`	indentação: recuo que mostra visualmente quais linhas pertencem a um bloco.

CÓDIGO COMENTADO

```
ALGORITMO – Somar itens do pedido da Hamburgueria Prime
(pseudocódigo: português estruturado, sem sintaxe de linguagem)

INÍCIO
  // ENTRADA: o que recebemos do cliente
  ler quantidade                 // ex.: 2
  ler precoUnitario              // ex.: 29.90 (Prime Clássico)

  // PROCESSO: decisões e cálculos
  se quantidade > 0 então        // condição: protege contra entrada inválida
    subtotal ← quantidade * precoUnitario
    se subtotal ≥ 100 então      // condição aninhada: regra de frete grátis
      entrega ← 0
    senão
      entrega ← 9.90
    fim se
    total ← subtotal + entrega
    mostrar "Total: R$ " + total // SAÍDA: o que o cliente vê
  senão
    mostrar "Quantidade inválida"
  fim se
FIM

CASOS DE TESTE (execute no papel antes de programar):
  entrada (2, 29.90)  → subtotal 59.80 + entrega 9.90 → "Total: R$ 69.70"
  entrada (0, 29.90)  → caminho do senão               → "Quantidade inválida"
  entrada (4, 34.90)  → subtotal 139.60 ≥ 100          → entrega 0 → 139.60
```

Leitura linha a linha

1. `INÍCIO ... FIM` — marcadores que delimitam a descrição do algoritmo. Escrever esses marcadores não garante a terminação: também é preciso verificar se todos os caminhos e laços realmente chegam ao fim.
2. `ler quantidade` — entrada: sem ela, o algoritmo só saberia resolver um caso fixo e não serviria para clientes diferentes.
3. `ler precoUnitario` — separar preço de quantidade permite reutilizar o mesmo algoritmo para os três itens do cardápio.
4. `se quantidade > 0 então` — condição de validação: o primeiro ponto onde um dado inválido é barrado antes de poluir o cálculo.
5. `subtotal ← quantidade * precoUnitario` — atribuição (seta, não igual): o resultado à direita substitui o valor da variável à esquerda.

6. `se subtotal ≥ 100 então ... senão ... fim se` — condição aninhada: repare no recuo mostrando que o bloco inteiro só existe dentro do primeiro `se`.
7. `entrega ← 0 / entrega ← 9.90` — o mesmo nome recebe valores diferentes conforme o caminho: é a variável mudando de estado.
8. `total ← subtotal + entrega` — soma sempre com a entrega já decidida; calcular antes de decidir seria a fonte clássica do erro "cobrou frete a quem tinha direito grátis".
9. `mostrar ...` — saída: o algoritmo só é útil se devolver algo observável.
10. `senão / fim se` — o caminho alternativo existe porque quantidade 0 ou negativa é possível; ignorar limites é como a maioria dos bugs nasce.

PRÁTICA GUIADA

1. Em papel ou bloco de notas, escreva o cabeçalho do algoritmo: nome, entrada (quantidade, preço unitário) e saída esperada (total formatado).
2. Digite no editor o pseudocódigo completo do bloco acima, mantendo os recuos — eles são a única "sintaxe" exigida.
3. Rode a simulação no papel com o caso `(2, 29.90)`: aponte o dedo em cada linha na ordem e anote o valor de cada variável ao lado.
4. Repita com `(0, 29.90)` e com `(4, 34.90)` e confira se bate com a tabela de casos de teste.
5. Altere a regra para "entrega grátis acima de R$ 80,00" e refaça os três testes — perceba que só as linhas da condição mudam.
6. **Resultado esperado:** um algoritmo versionado na pasta do projeto (ex.: `notas/algoritmo-pedido.txt`) com os três casos de teste e os valores intermediários anotados; evidência: captura do arquivo com os testes conferidos.

EXERCÍCIOS

1. **FIXAÇÃO** Quando uma condição é mais adequada que uma repetição? Cite um exemplo de cada um usando o cardápio.

   ▼ Gabarito e dica

   Condição decide qual caminho executar: "se o subtotal é maior ou igual a R$ 100, entrega grátis". Repetição executa um bloco várias vezes: "para cada item do pedido, somar ao total". Dica: decidir entre caminhos é diferente de repetir uma ação. Uma repetição pode ter quantidade conhecida de antemão ou depender de uma condição; saber o número de vezes não a transforma em uma condição isolada.

2. **APLICAÇÃO** Escreva o algoritmo que soma dois itens do cardápio (ex.: 1 × Prime Clássico R$ 29,90 + 1 × Prime Veggie R$ 32,90) e mostra o subtotal.

   ▼ Gabarito e dica

Esperado: entrada das duas quantidades e dois preços; `subtotal1 ← qtd1 * 29.90`, `subtotal2 ← qtd2 * 32.90`, `total ← subtotal1 + subtotal2`, saída "Subtotal: R$ 62.80" para 1+1. Dica: use variáveis com nomes que digam o que guardam, não `x` e `y`.

3. **FIXAÇÃO** O que acontece se o algoritmo acima não tratar o caso `quantidade = 0`? E se a entrada for negativa?

   ▼ Gabarito e dica

   Sem a validação, mantendo o restante do cálculo, quantidade 0 produziria subtotal 0 e total R$ 9,90, pois a entrega continuaria sendo cobrada. Quantidade negativa produziria um subtotal negativo e um total incorreto, que poderia até ser negativo dependendo dos valores. Isso não significa que algum pagamento seria executado: o exemplo só calcula e mostra um valor. Por isso existe o `se quantidade > 0` com caminho alternativo de mensagem. Dica: teste entradas inválidas antes de aceitar o pedido.

4. **DESAFIO** Proponha um algoritmo para "escolher o item mais caro do pedido" entre os três do cardápio, sem usar repetição — e explique por que a solução fica feia.

   ▼ Gabarito e dica

   Dá para comparar com duas condições aninhadas: guardar o maior, comparar com o 2º, depois com o 3º. Fica feio e não escala: com 20 itens, o código explode. É exatamente o exemplo que motiva a repetição (`para cada item`) da próxima aula. Dica: se o número de comparações cresce com os dados, você precisa de um laço.

ERROS COMUNS

- **Confundir `=` de igualdade com atribuição:** no pseudocódigo usamos `←` ou `=` com sentido único; aluno escreve `total = subtotal + entrega` e não sabe se está perguntando ou guardando. Explique os dois papéis antes de ver a sintaxe real.
- **Esquecer o `fim se` ou o recuo:** o bloco passa a engolir linhas seguintes e a simulação diverge. Corrija pedindo que leia em voz alta "isto aqui está dentro do quê?".
- **Testar só o caso feliz:** todos os exemplos com quantidade 2 e ninguém testa 0. Exija sempre um caso-limite e um caso inválido na prática.
- **Usar nomes de variável sem significado (`a`, `x1`):** três linhas depois ninguém sabe o que `x1` guarda. Troque por `subtotal`, `quantidade`, `entrega`.
- **Começar a digitar código antes do plano:** o aluno abre o `app.js`, escreve meio algoritmo e trava. Combine a regra: primeiro o papel, depois o editor.

CRITÉRIOS DE VERIFICAÇÃO

- ✓ O estudante explica com as próprias palavras a diferença entre variável, condição e repetição, com um exemplo do cardápio para cada.
- ✓ O algoritmo do pedido está completo, com entrada, processo, saída e recuos corretos, e produz os totais esperados nos três casos de teste.
- ✓ A atividade tem evidência verificável: arquivo do pseudocódigo com os valores intermediários anotados na simulação manual.
- ✓ O estudante aponta uma limitação: por exemplo, o algoritmo não trata preço negativo ou precisa de repetição quando o número de itens é variável.

← Aula anterior: Ferramentas e organização

Próxima aula: Acessibilidade e ética digital →

AULA 04

Acessibilidade e ética digital

Iniciante | 50 min | Projeto: critérios de acessibilidade e privacidade do cardápio

---

PARA O PROFESSOR

1. **Diagnóstico (5 min):** pergunte: "como você faria o pedido pelo telefone se fosse cego?" As respostas costumam revelar que ninguém pensou no leitor de tela — aproveite o desconforto.
2. **Demonstração (10 min):** mostre a navegação por Tab em um site real: o foco some ao chegar em botões feitos com `div` e reaparece em botões de verdade. Mostre também um contraste ruim (texto cinza claro) e o mesmo texto corrigido.
3. **Programação guiada (15 min):** escrevam juntos o texto alternativo de três fotos do cardápio e discutam alternativas para cada uma — é a parte que mais gera debate produtivo.
4. **Prática individual ou em duplas (15 min):** cada dupla monta a lista de decisões de acessibilidade e privacidade da Hamburgueria Prime, testando um item real (Tab, contraste ou leitor de tela do celular).
5. **Fechamento (5 min):** pergunte: "por que texto alternativo não pode ser apenas 'imagem'?" e registre as definições da turma no caderno.

Dica: conecte sempre com bolso: site inacessível perde cliente e pode gerar passivo legal (Lei Brasileira de Inclusão); dado coletado sem necessidade vira risco de vazamento — e, para o projeto fictício, a regra prática é "se não vamos usar, não vamos pedir". Se não houver computador, faça o teste do leitor de tela do próprio celular (TalkBack/VoiceOver) sobre o site da turma.

EXPLICAÇÃO

Acessibilidade é projetar para quem usa o site de formas diferentes das suas. Uma parte relevante dos usuários navega com leitor de tela (pessoas cegas ou com baixa visão), outros apenas com o teclado (lesão motora, repetitivo), outros dependem de contraste alto, tamanho de texto ampliado ou navegação lenta por limitações cognitivas. Nada disso é "versão para deficientes": é a mesma página, interpretada por caminhos diferentes. Quando a estrutura é bem feita, todos esses caminhos funcionam; quando ela é gambiarra visual, o leitor de tela lê uma bagunça e o teclado nem chega ao botão.

Cinco decisões tomadas desde o primeiro dia resolvem a maior parte dos problemas: navegação por teclado com foco visível; texto alternativo descritivo em toda imagem informativa; contraste suficiente entre texto e fundo; hierarquia de títulos que faça sentido (um `h1`, depois `h2`); e linguagem clara, sem jargão desnecessário nas instruções de pedido. O plano é barato na fase de esboço e caríssimo depois — reescrever HTML inteiro para consertar hierarquia custa muito mais do que escrever certo desde a aula 08.

O texto alternativo merece atenção dupla porque é o erro mais comum. Ele descreve a **informação** da imagem, não a imagem: `alt="Hambúrguer clássico com queijo, salada e molho da casa"` diz ao ouvinte o que ele perderia ao não ver. Escrever "imagem" ou deixar vazio em imagem informativa equivale a entregar um cardápio em branco. Já em imagem decorativa, o vazio (`alt=""`) é a escolha correta: ele diz ao leitor de tela para pular. E a regra do curso é reforçada aqui: as fotos do cardápio só entram com descrição escrita junto — o texto alternativo não se inventa na hora da entrega.

Ética digital é a face administrativa do mesmo cuidado. A Hamburgueria Prime não precisa do nome da mãe, da data de nascimento nem do CPF do cliente para vender um Prime Cheddar: colecionar dados "porque pode ser útil" aumenta o custo de proteção, o risco de vazamento e a desconfiança. A LGPD reforça isso legalmente, mas o princípio é anterior à lei: só se coleta o que se usa, só se usa para o que se combinou, e o cliente pode pedir para sair da lista. Para um projeto de curso, isso vira decisões concretas — formulário com campos mínimos, aviso claro de uso dos dados e nenhuma chave de API ou telefone real embutido em repositório público.

Conectando com a aula anterior: algoritmos também têm ética. Um cálculo de entrega que esconde taxas, um preço que muda conforme o perfil do cliente ou um botão que engana o usuário a assinar algo são decisões de projeto, não apenas de código. Gravar esses critérios agora, no repositório da Hamburgueria Prime, cria um padrão que você pode auditar a cada nova funcionalidade — e é exatamente assim que times profissionais trabalham: acessibilidade e privacidade entram na definição de pronto, não na revisão final.

VOCABULÁRIO TÉCNICO

TERMO (EN)	SIGNIFICADO (PT-BR)
`accessibility` (a11y)	acessibilidade: tornar o conteúdo utilizável por qualquer pessoa, independente de limitação.
`alt text`	texto alternativo: descrição da informação de uma imagem para quem não a vê.
`keyboard navigation`	navegação por teclado: percorrer a página com Tab/Shift+Tab e ativar com Enter/Espaço.

TERMO (EN)	SIGNIFICADO (PT-BR)
`focus`	foco: indicador visual de qual elemento receberá a próxima ação do teclado.
`contrast`	contraste: diferença de luminosidade entre texto e fundo; baixo contraste dificulta a leitura.
`screen reader`	leitor de tela: programa que converte a página em fala ou Braille.
`privacy` (privacidade)	privacidade: controle do usuário sobre quais dados são coletados e para que servem.
`LGPD`	Lei Geral de Proteção de Dados brasileira: regras de coleta, uso e consentimento de dados pessoais.

CÓDIGO COMENTADO

```
<!— FOTO INFORMATIVA: o alt carrega a informação que a imagem transmite —>
<img src="assets/img/classico.webp"
     alt="Prime Clássico: pão brioche, carne 180 g, queijo prato, salada e molho da casa" >

<!— FOTO DECORATIVA: alt vazio manda o leitor de tela pular —>
<img src="assets/img/ondas-decorativas.webp" alt="" >

<!— BOTÃO DE VERDADE: navegável por teclado e anunciado como ação —>
<button type="button" onclick="adicionar('Prime Cheddar')">
  Adicionar Prime Cheddar — R$ 34,90
</button>

<!— LINK: navega para outro lugar; rótulo diz o destino, não "clique aqui" —>
<a href="#cardapio">Ver o cardápio completo</a>

<!— CAMPO DE FORMULÁRIO: rótulo associado ao campo (fundamento da Aula 11) —>
<label for="telefone">Telefone para contato</label>
<input id="telefone" name="telefone" type="tel"
       autocomplete="tel"
       placeholder="Somente o que precisamos para o pedido">

<!— PRIVACIDADE: não pedir o que não será usado —>
<!— ERRADO: <input name="cpf">  <input name="data-nascimento">  (nada disso é usado) —>
```

Leitura linha a linha

1. `alt="Prime Clássico: pão brioche, ..."` — descreve o produto com detalhe que importa ao cliente; sem isso, o leitor de tela anuncia só "imagem".
2. `alt=""` — vazio de propósito: sinaliza que a imagem não agrega informação e deve ser ignorada; deixar ausente faz alguns leitores anunciarem o nome do arquivo.
3. `<button type="button">` — elemento nativo: recebe foco, Tab, Enter e anúncio de ação sem nenhum JavaScript de acessibilidade extra.
4. `Adicionar Prime Cheddar – R$ 34,90` — o texto do botão descreve a ação e o resultado; "clique aqui" não diz nada a quem escuta.
5. `<a href="#cardapio">Ver o cardápio completo</a>` — o rótulo promete o destino: quem usa leitor de tela lista todos os links antes de escolher um.
6. `label ... for="telefone"` — o atributo `for` liga o rótulo ao campo: clicar no texto foca o input e o leitor lê nome + instrução juntos.
7. `type="tel"` — teclado numérico/telefônico no celular; detalhe pequeno que economiza erro de digitação.
8. `autocomplete="tel"` — permite o preenchimento automático do próprio aparelho; menos atrito, menos erro, e funciona a favor do usuário.
9. `<!— ERRADO: cpf, data-nascimento —>` — comentário didático do que não fazer: dado não usado é só risco acumulado (LGPD).

PRÁTICA GUIADA

1. Liste as imagens previstas do projeto (logo, Prime Clássico, Prime Cheddar, Prime Veggie) e escreva uma frase de texto alternativo para cada uma, mencionando o produto e o essencial da foto.
2. Em um site qualquer, percorra a página inteira usando apenas Tab: anote onde o foco some e onde ele aparece; identifique os elementos responsáveis (geralmente `div` usado como botão).
3. Teste o contraste de um texto do seu projeto (ferramenta de contraste ou inspeção visual): anote a combinação reprovada e a corrigida.
4. Escreva a lista de decisões da Hamburgueria Prime: campos do formulário que realmente serão usados (nome, telefone, endereço de entrega), o que nunca será pedido (CPF, data de nascimento) e qual regra de privacidade será exibida.
5. Ative o leitor de tela do celular (TalkBack no Android, VoiceOver no iPhone) sobre a página que a turma estiver usando e escute como o cardápio é lido em voz alta.
6. **Resultado esperado:** um documento `notas/acessibilidade-e-privacidade.md` com os quatro alts escritos, a lista de campos do formulário e duas melhorias de contraste/foco identificadas; evidência: captura do teste de Tab e do leitor de tela.

EXERCÍCIOS

1. **FIXAÇÃO** Por que o texto alternativo não deve ser apenas "imagem"? Escreva um substituto adequado para a foto do Prime Veggie.

   ▼ Gabarito e dica

Porque o alt transmite a informação que a imagem carrega; "imagem" acrescenta zero e obriga o ouvinte a adivinhar. Exemplo aceito: `alt="Prime Veggie: burger de grão-de-bico com queijo, alface e tomate"`. Dica: pergunte "o que este cliente precisa saber para pedir?" e responda em uma frase.

2. **APLICAÇÃO** Aponte quais dados um formulário de pedido da Hamburgueria Prime deve pedir e quais não deve, justificando cada caso.

   ▼ Gabarito e dica

   Devem: nome (para dirigir o pedido), telefone/WhatsApp (confirmar pedido), endereço (entrega), itens e observações (alergia). Não devem: CPF, data de nascimento, dados de cartão no front-end, nome da mãe — nada disso é necessário para vender um hambúrguer e cria risco. Dica: justifique com a pergunta "para que usaremos?" — sem resposta, sem campo.

3. **FIXAÇÃO** Qual é a diferença entre um botão feito com `div` e um feito com `<button>` para quem navega por teclado?

   ▼ Gabarito e dica

   O `<button>` nativo já recebe foco, responde Enter e Espaço e é anunciado como botão pelo leitor de tela; a `div` não faz nada disso sem JavaScript e estilos extras, e some do percurso por Tab — usuário de teclado simplesmente não alcança a ação. Dica: cite a demonstração do Tab da aula.

4. **DESAFIO** Revise uma página do projeto ou de um site conhecido e produza um relatório com três falhas de acessibilidade e três de privacidade, cada uma com a correção proposta.

   ▼ Gabarito e dica

   Relatório com pares "problema → correção": ex.: alt ausente → escrever descrição do produto; foco invisível → manter o indicador nativo não remover com `outline: none`; pedido de dado não usado → remover campo e explicar o uso de cada dado restante. Dica: para cada item, registre como você verificou (Tab, leitor de tela, inspeção do formulário) — evidência do teste vale mais que a opinião.

ERROS COMUNS

- `alt="imagem"` **ou** `alt="foto do hambúrguer"` **genérico:** descreve o óbvio e desperdiça a única chance de informar. Corrija reescrevendo o que a foto mostra de relevante para pedir.
- **Usar** `div` **como botão/link:** some do Tab e não é anunciado como ação. Troque por `<button>` ou `<a>` conforme o comportamento (ação vs. navegação).

- **Remover o foco com `outline: none` "porque fica feio":** torna a página impossível para quem usa teclado. Se o foco padrão incomodar, desenhe outro visível — nunca o remova sem substituir.
- **Contraste baixo (cinza claro sobre branco):** parece elegante e ilegível em telas pequenas ao sol. Aumente o contraste antes de discutir fontes.
- **Pedir dados "para o futuro":** CPF, nascimento e dados de cartão sem necessidade violam o princípio da necessidade e ampliam o dano possível em caso de vazamento. Corte campos sem uso.

CRITÉRIOS DE VERIFICAÇÃO

- ✓ O estudante explica com as próprias palavras por que acessibilidade e privacidade entram desde o início e não na revisão final.
- ✓ O documento de decisões existe com quatro textos alternativos escritos, a lista de campos permitidos/proibidos e ao menos duas verificações realizadas.
- ✓ A atividade tem evidência verificável: captura do teste de navegação por Tab e do leitor de tela lendo o cardápio.
- ✓ O estudante aponta uma limitação/cuidado: contraste, foco, uso de dado pessoal ou coerência com a LGPD — citando o que faria diferente no próximo módulo.

← Aula anterior: Algoritmos e lógica

Fim do módulo: siga para o Módulo 02 →

Checkpoint do módulo

Ao final do Módulo 01, o projeto Hamburgueria Prime deve ter:

- Pasta `hamburgueria-prime/` criada com a árvore `index.html`, `css/`, `js/` e `assets/img/`, usando nomes minúsculos, sem acento e sem espaço.
- O desenho da viagem de uma requisição (5 passos) com os desvios 404, 500 e offline marcados, guardado nas notas do projeto.
- O algoritmo do cálculo do pedido em pseudocódigo, com os três casos de teste (normal, zero e quantidade grande) conferidos manualmente.
- O documento de acessibilidade e privacidade com os textos alternativos do cardápio e a lista de campos que o formulário poderá pedir.
- Um servidor local funcionando na raiz do projeto, pronto para receber o HTML das aulas do Módulo 02.